

ERC Starting Grant 2026

ectype – a copy without an original

ECTOCINEMA

New Topologies of Cinematic Time

Dr Daniel Chávez Heras

King's College London · 60 months

From recorded to predicted temporalities...



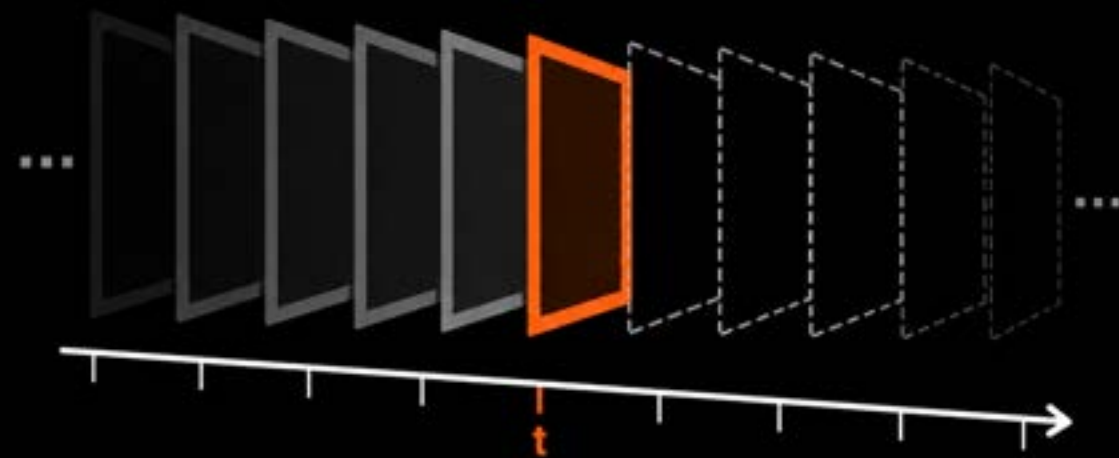
1900



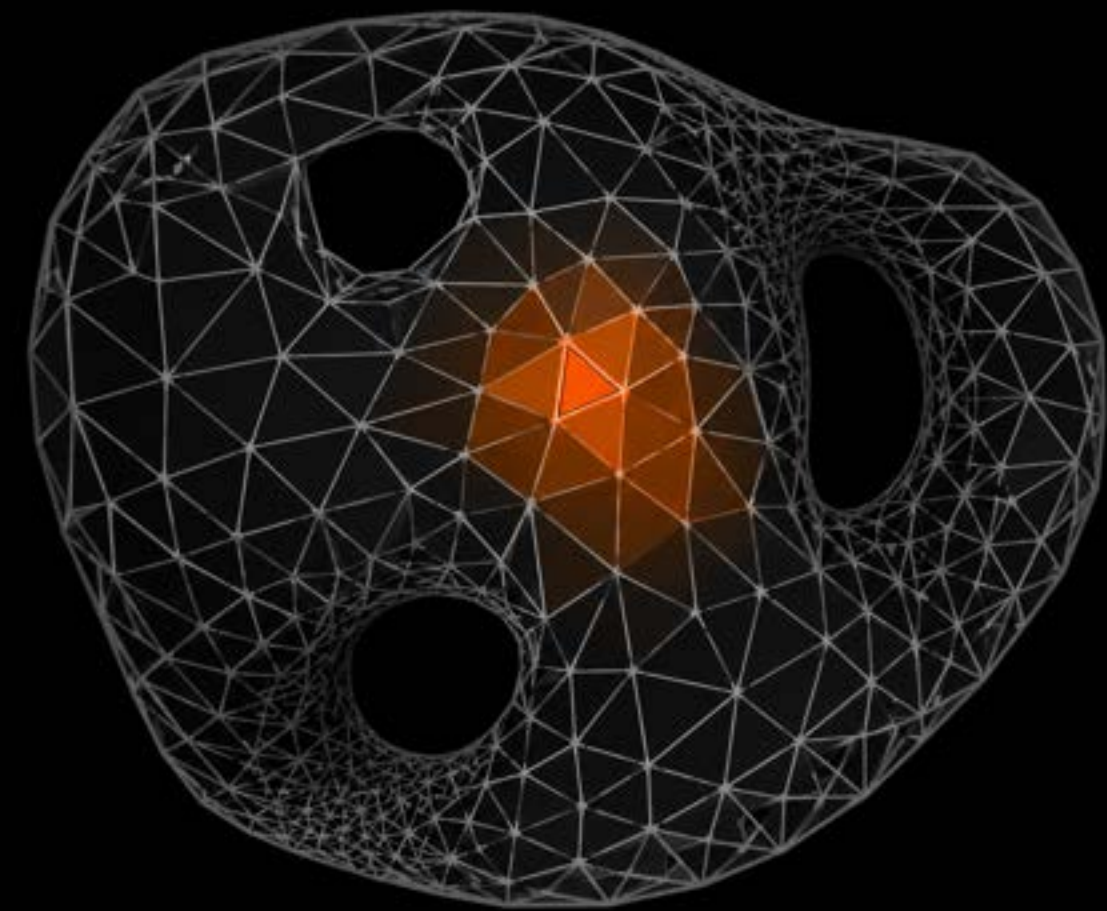
2024

...a computational model of cinematic time.

What if time had a different shape?



TIMELINE



TIME-MESH



Rethinking
screen archives



Reshaping
editing interfaces



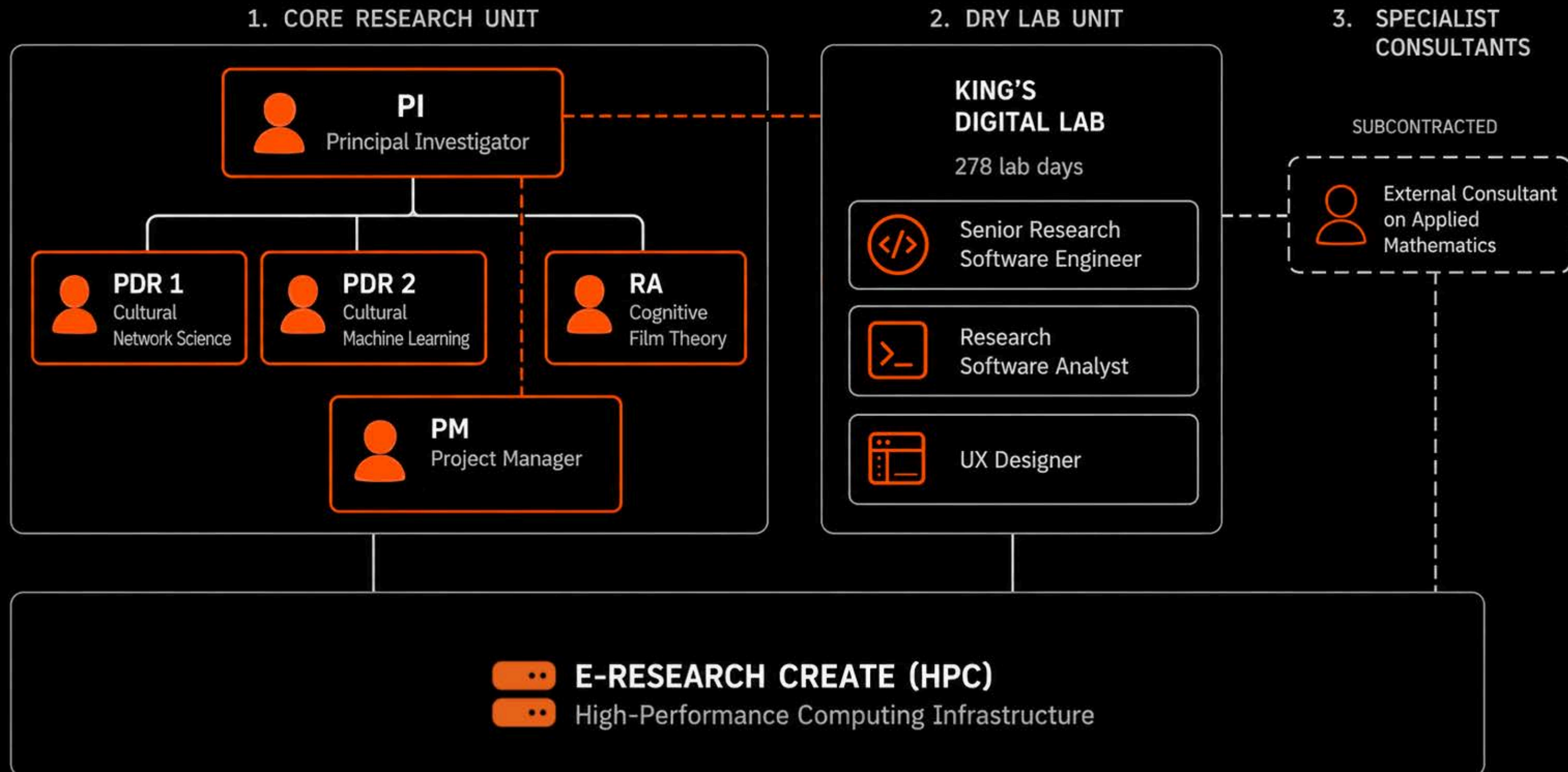
Redrawing
disciplinary
boundaries

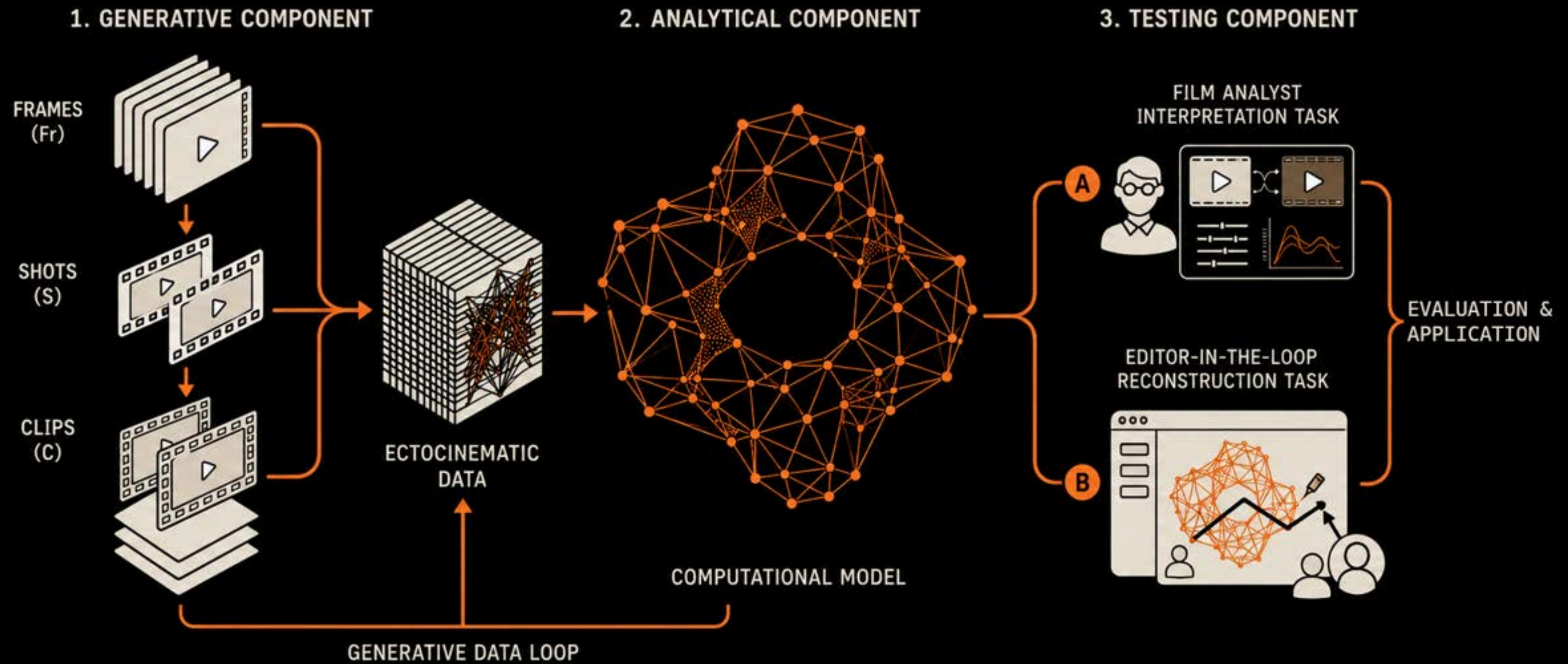
From film and video analytics, to **growing synthetic screen cultures** in the lab.

ECTOCINEMA

New Topologies of Cinematic Time

End of presentation





README

MIT license

FrameSense

FrameSense is a highly modular command line tool to pre-process your video collections.

Status: alpha

Current focus: functional minimum viable product. The current implementation is still buggy and slow. Consolidation and optimisation will come at a later stage.

Requirements

- python 3.10+
- Docker or Singularity

Initial set Up

- Clone this repository anywhere on your system;
- Copy `docs/collections.json` to the folder containing your video collections and adapt its contents to your needs. All paths `collections.json` are relative to that file.
- Copy `docs/.env` in the folder that contains `framesense.py` and adapt the values to your environment. `FRAMESENSE_COLLECTIONS` should be the absolute path to your copy of `collections.json`.
- Create the python virtual environment: `python3 -m venv venv`
- Activate the virtual environment: `source venv/bin/activate`
- Install the required packages: `pip install -r requirements.txt`

Concepts

In FrameSense your collection is broken down into a hierarchy of smaller units. A `Collection` contains `videos`, which are made of `clips`. Each Clip is a sequence of `shots`. And each Shot is composed of a series of still `Frames`.

Each operation provided by FrameSense works at on a particular unit in that hierarchy.

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors

- geoffroy-noel-ddh Geoffroy Noël
- chavezheras Daniel
- alder-chat-bot Alder

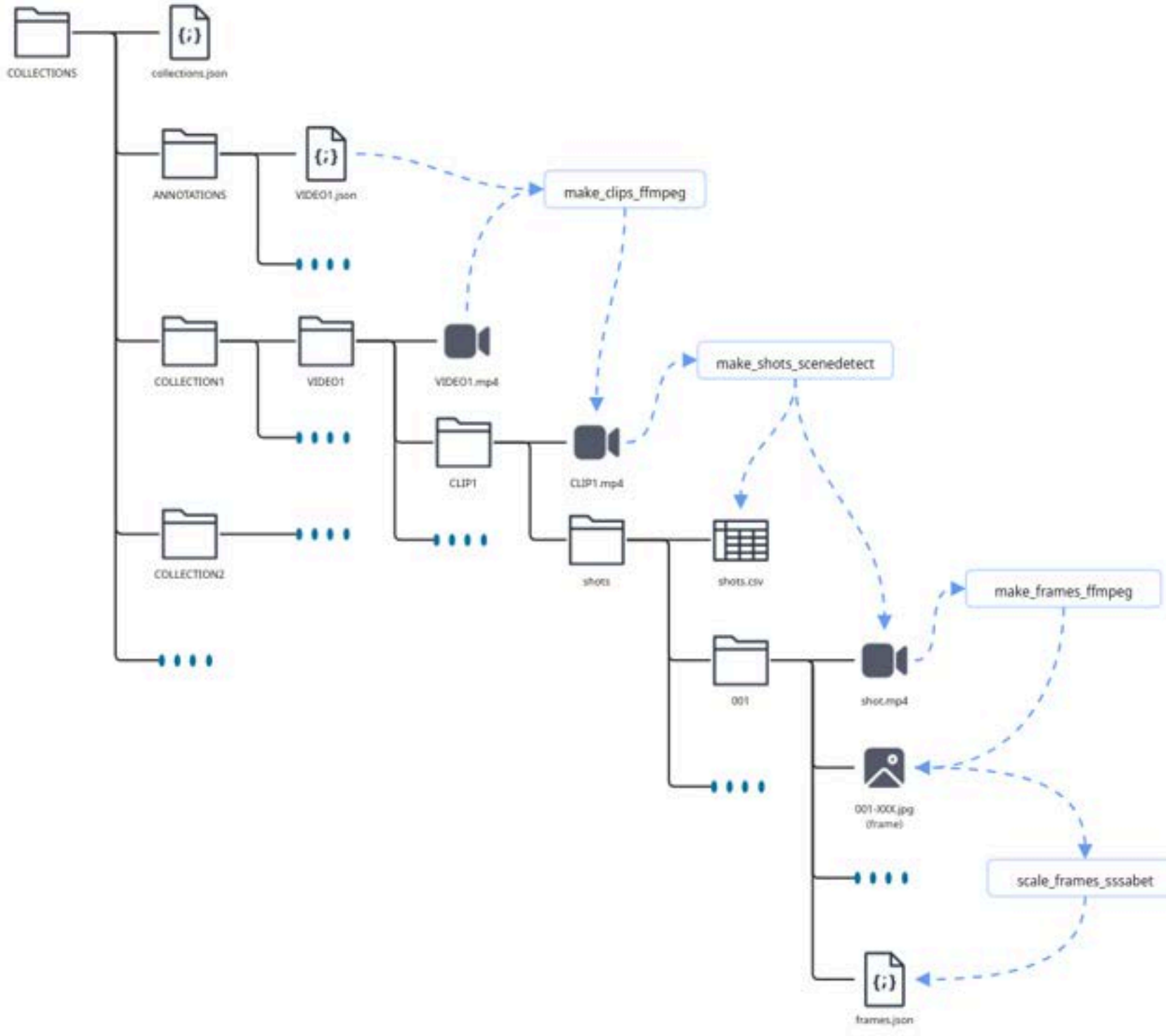
Languages

Python 93.0% Dockerfile 5.2% Shell 1.8%

Suggested workflows

Based on your tech stack

- [Python application](#) [Configure](#)
Create and test a Python application.
- [Python package](#) [Configure](#)
Create and test a Python package on multiple Python versions.
- [Publish Docker Container](#) [Configure](#)
Build, test and push Docker image to GitHub Packages.





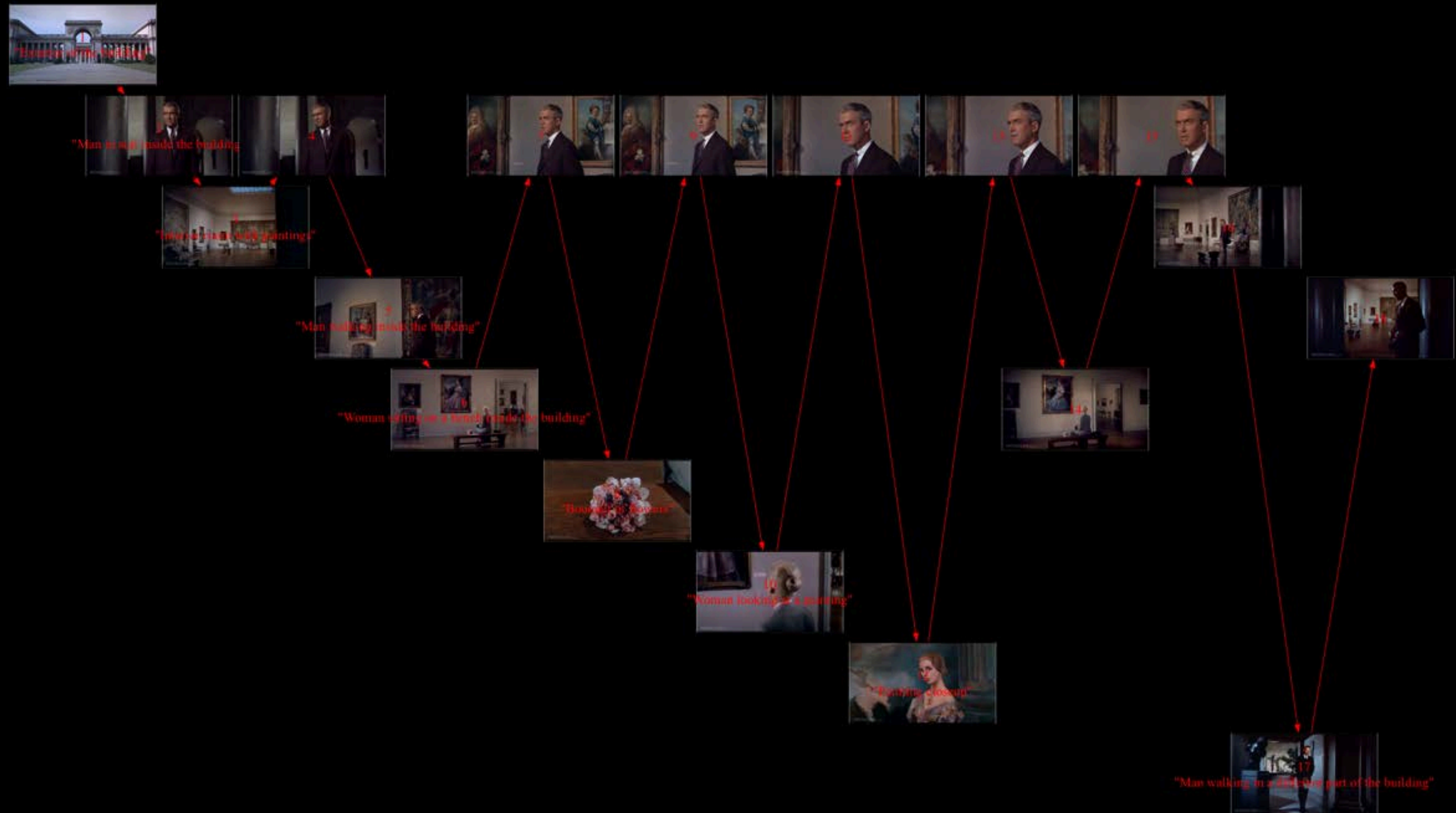
SCULPTING TIME WITH COMPUTERS EXPERIMENTS

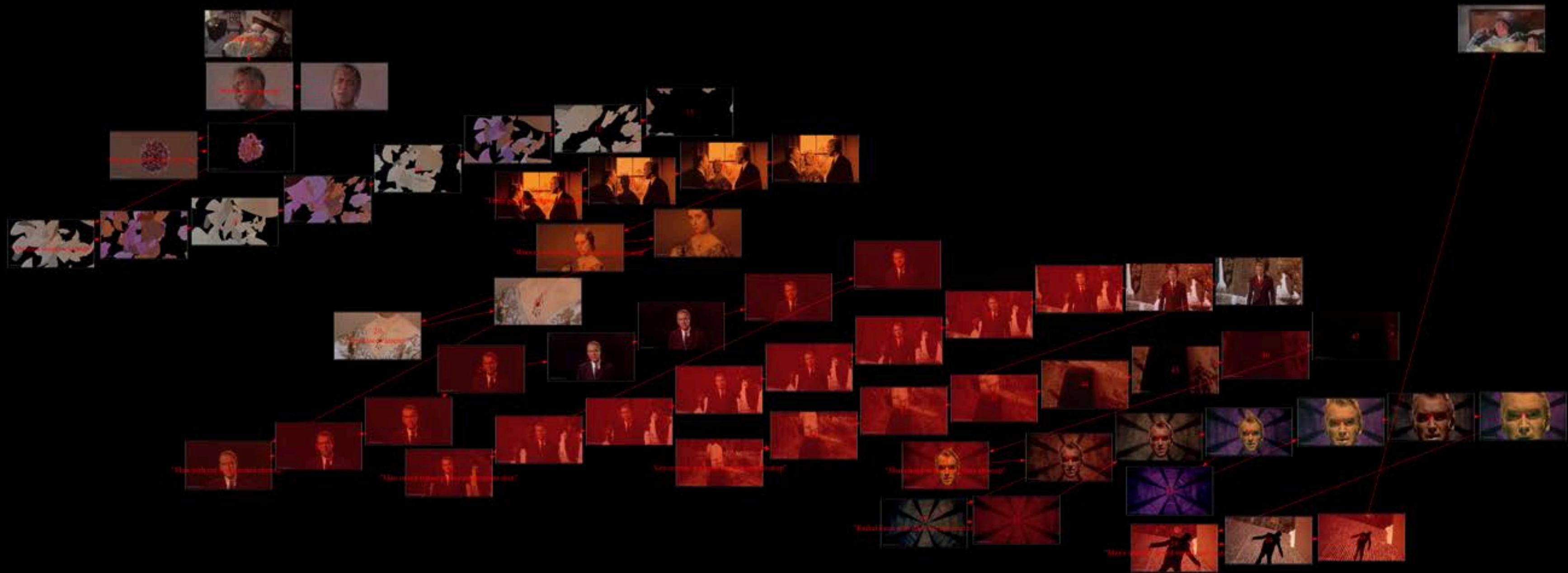
ADDITIONAL SLIDES

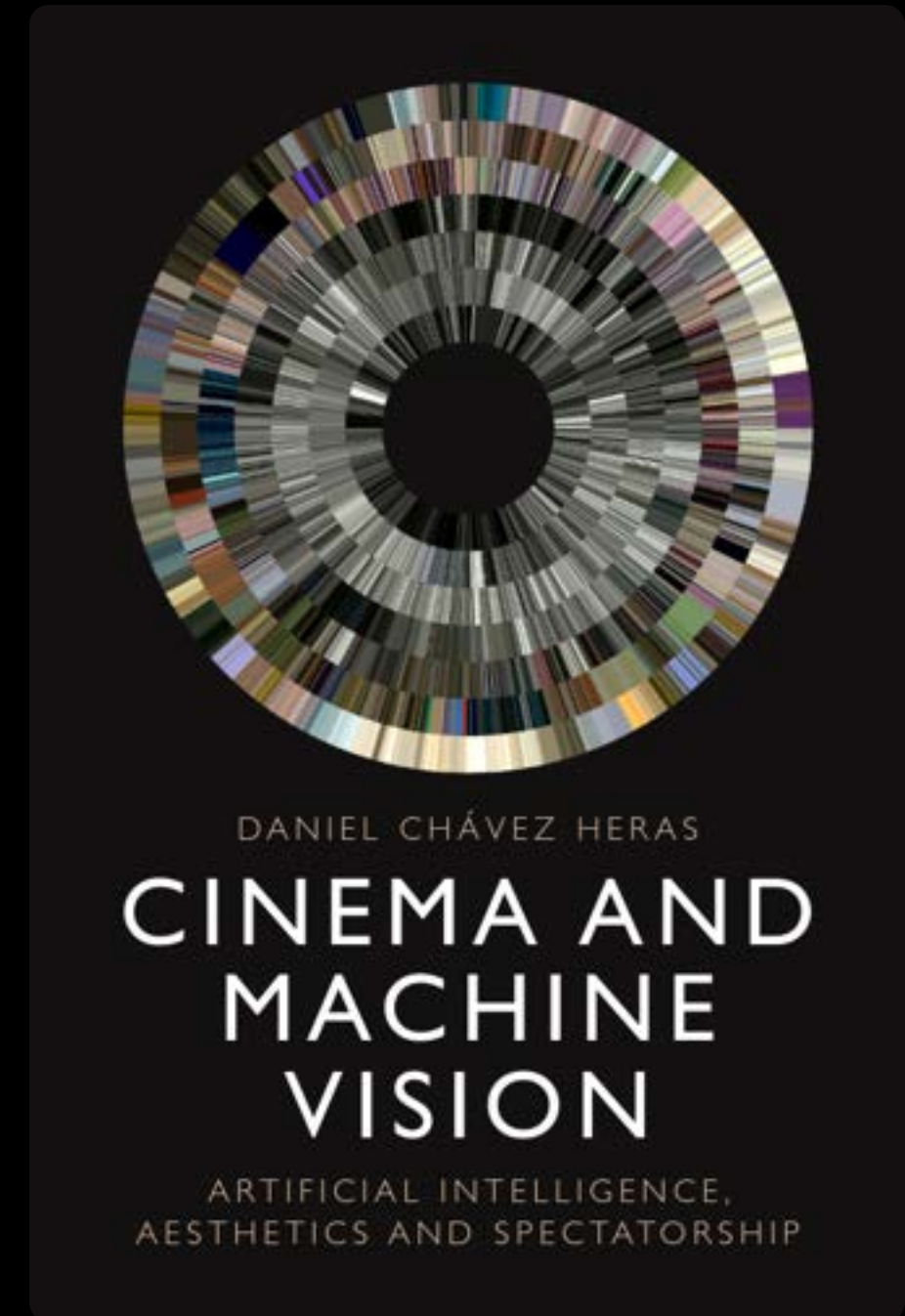
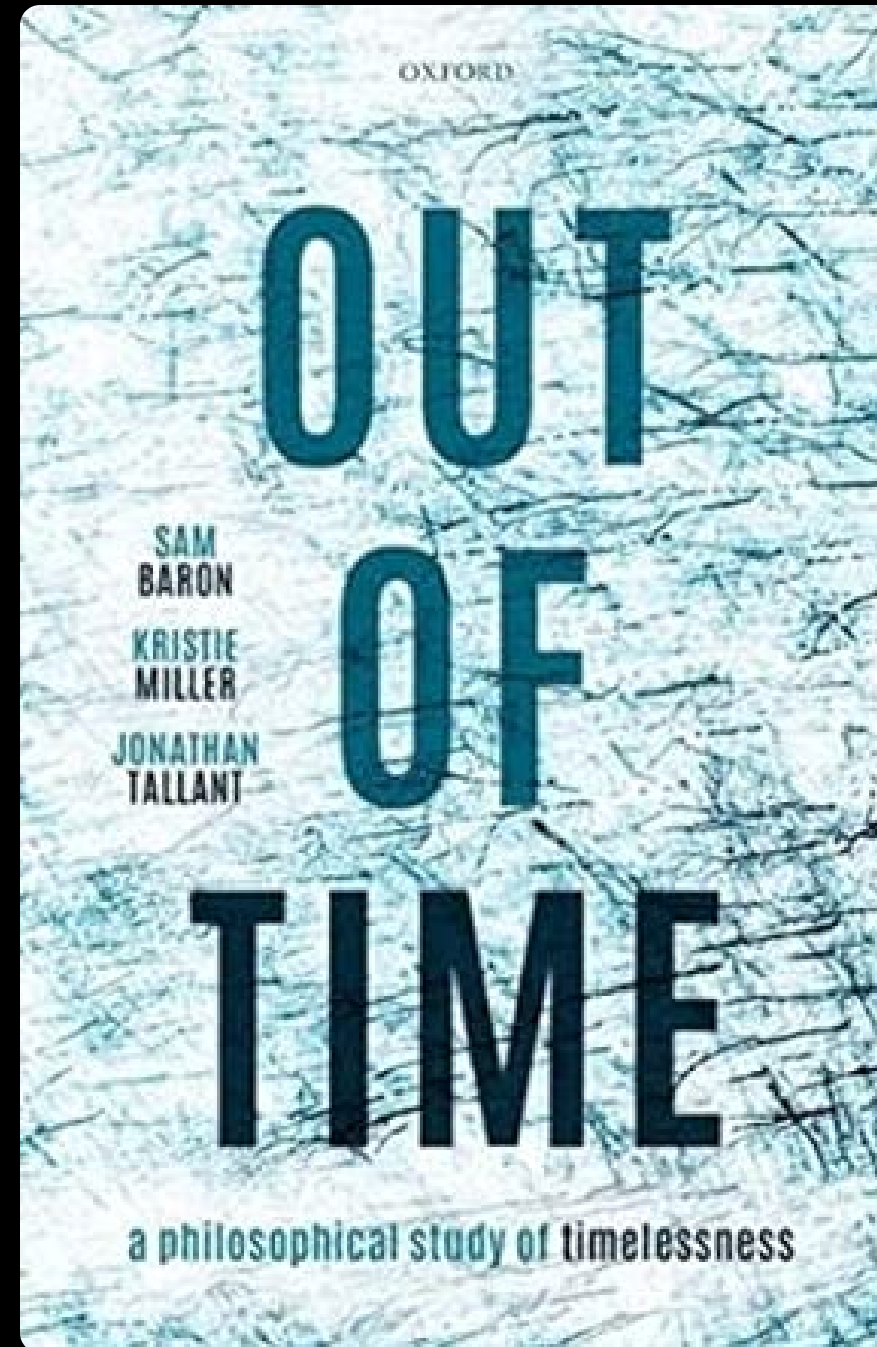
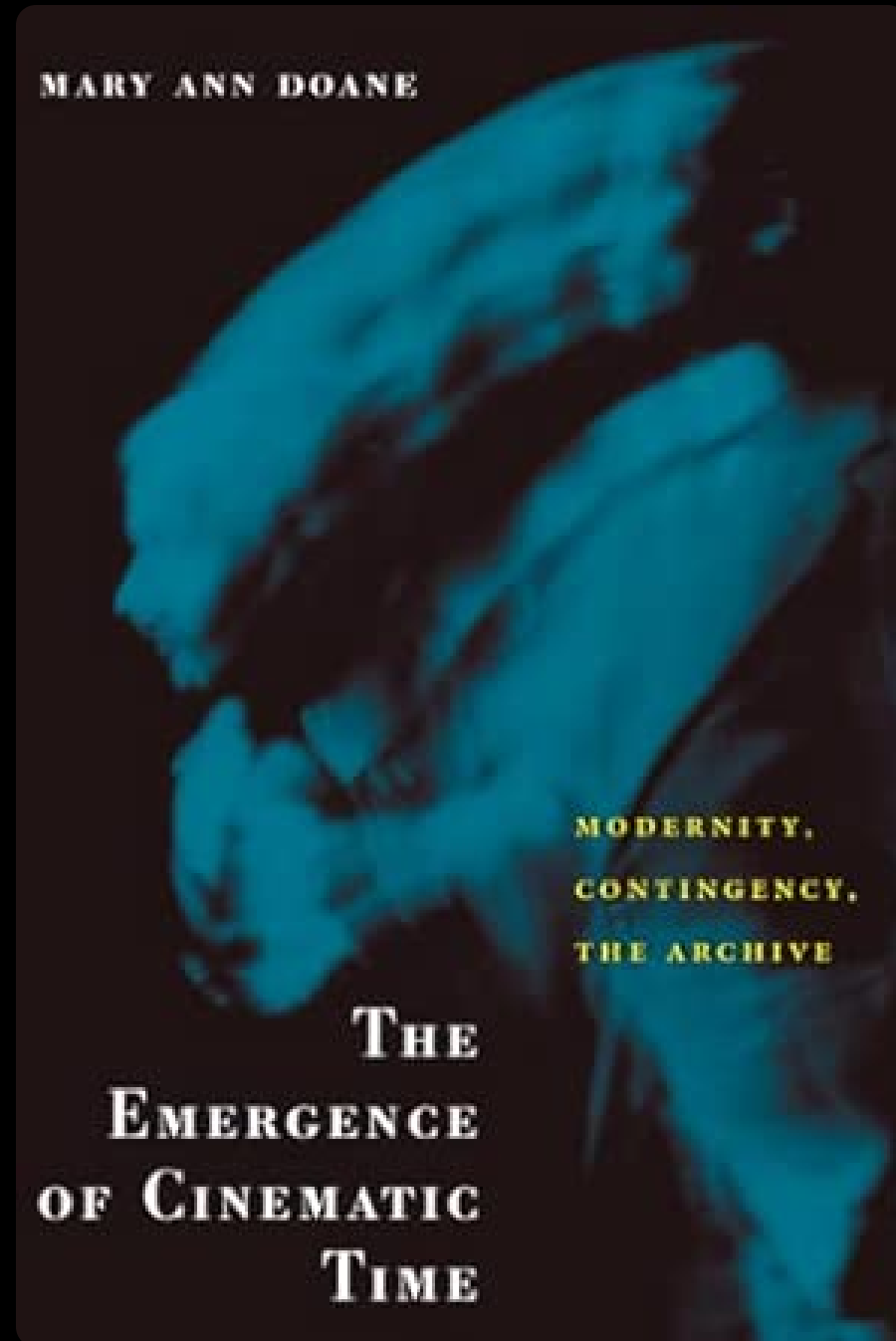


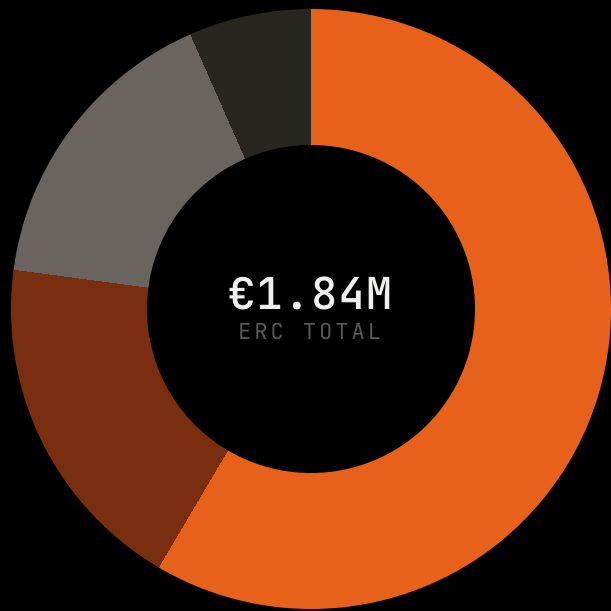
SCULPTING TIME WITH COMPUTERS EXPERIMENTS

ADDITIONAL SLIDES









- Personnel 58.5% €1,078k
- Dry Lab (KDL + HPC) 18.6% €342k
- Indirect costs 16.3% €300k
- Travel, publications & other 6.6% €122k

PERSONNEL ALLOCATION - 60 MONTHS

